

Insertion - Algorithm All Questions

Question #01

// Simple just loops & Explained in DRY_RUN

(DRY_RUN)

```
void bubblesort(int arr[], int n)
{
    for (int i = 0; i < n-1; i++)
```

Suppose:
our arr is
{7, 2, 6, 3}
n = 4

bool swapped = false;

```
    {
        for (int j = 0; j < n-1; j++)
```

```
        {
            if (arr[j] > arr[j+1])
```

```
            {
                int t = arr[j];
                arr[j] = arr[j+1];
```

t = 7
arr[0] = arr[1]
arr → {2, 2, 6, 3}

```
                arr[j+1] = t;
                swapped = true;
            }
        }
    }
```

arr → {2, 7, 6, 3}

⑥ again

```
    for (int j = 0; j < n-1; j++)
```

```
    {
        if (arr[j] >
```

```
            arr[j+1])
                swap;
```

arr → {2, 6, 7, 3}

⑥ again for (—) 2 < 3 ✓

{
if (—) 7 > 3 ✓

{
swap; arry → {2, 6, 3, 7}
}

⑦ for (—) 3 < 3 ✗

if (!sw) ✗

again for (—) 1 < 3

⑧ for (int j=0; —) 0 < 3 ✓

{
if (—) 2 > 6 ✗

⑨ again for (—) 1 < 3 ✓

{
if (—) 6 > 3 ✓

{
swap; → arry → {2, 3, 6, 7}

⑩ again for (—) 2 < 3 ✓

if (—) 6 > 7 ✗

if (!sw) ✗



continue until loops end.

Question #1

1 Explained in DR
(DRY RUN)

Suppose: our arr is:
{ 3, 2, 7, 6 } $n=4$

void SS(int* arr, int n)

{
 a for(int i=0; i<n-1; i++) OK ✓
 {
 int mi=i; mi=0

b for(int j=i+1; j<n; j++) OK ✓
 {
 if(arr[j]<arr[mi]) ✓ 2<3
 {
 mi=j; mi=1
 }
 }

b again: for(—) ✓ $j=2 < 4$

if(—) ✗ $6 < 3$

for(—) ✓ $j=3 < 4$

if(—) ✗ $7 < 3$

if(mi != i) {

int t = arr[i];

swap: }

[i=0, mi=1] → swap

arr → { 2, 3, 7, 6 }

Question # 03

(DRY-RUN)

suppose arr is: { 3, 2, 7, 6 }, n=4

void ins (int arr[], int n)

{
for (int i=1; i<n; i++) 1 < 4 ✓

{
int key = arr[i]; arr[i] = 2
int j = i-1; j = 0

while (j >= 0 && arr[j] > key) ✓ 0 >= 0 && 3 > 2

{
arr[j+1] = arr[j]; → arr = {3, 3, 7, 6}
j--;

while (—) ✗ j = -1 + 1 = 0
arr[j+1] = key; → arr = {2, 3, 7, 6}

}

again for (—) ✓ 2 < 4

int k = arr[i] k = 7

int j = i-1; j = 1

while (—) ✗ 3 > 7

again for (—) ✓ 3 < 4

int k = arr[i] k = 6

int j = i-1; j = 2

while (—) ✓ 7 > 6

↓
continue

Question #04

// Explained in DR: (DAY RUN)

Supposes Our arr is: {3, 2, 7, 6}

*) (a)

void mergesort(int arr[], int L, int R)

{
if (L < R) ✓ 0 < 3

{
int mid = L + (R - L) / 2; 0 + (3) = 1.5 → 1
mid = 1
mergesort(arr, L, mid);

(b) mergesort(int* arr, int L, int R)
if (L < R) ✓ 0 < 2
{

int m = L + (R - L) / 2; m = 1
mergesort(arr, L, mid);

(c) mergesort(int* arr, int L, int R)
if (L < R) ✓ 0 < 1
{

int m = L + (R - L) / 2; m = 0
mergesort(arr, L, mid);

(d) ms(int*)
if (L < R) ✗ 0 < 0

merges(arr, mid, right);

(e) ms(arr, 1 →)

if (L < R) 1 < 4

merge(arr, L, mid, right);

01 void merge(int arr, int L, int M, int R)

int n1 = M - L + 1; n1 = 1
int n2 = R - M; n2 = 1

int LA[n1];
int RA[n2];

✓ 0 < i < n1 ← for (int i = 0; i < n1; i++)

LA[i] = arr[L+i]

✓ 0 < i < n2 ← for (int i = 0; i < n2; i++)

RA[i] = arr[M+1+i]

int i = 0;
int j = 0;
K = 0 ← int k = Left; K = 0

✓ while (i < n1 && j < n2)

✗ if (arr[i] < arr[j])

arr[k] = arr[i];

else

arr[k] = arr[j];

✓ arr[k] = 3
j = 1

k++;

k = 1


```

0 < 1 ← while (i < n1)
{
    arr[i] = 3 ← arr[k] = LAC[i]
    i++;      i++;
    k++;      k++;
}

```

~~IX~~ ← while (j < n2)

return to (b)

continue

Question # 05

// Explained in DR:

(DRY RUN)

void quicksort(int* arr, int L, int H) Suppose
arr is
{3, 2, 7}
size = 3

if (L < H) ✓ 0 < 2

int p = partition(arr, L, H);

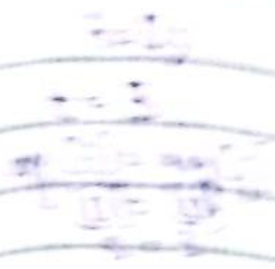
partition(int* arr, int L, int H)

piv = 7 ← int piv = arr[high];
i = -1 ← int i = low - 1;

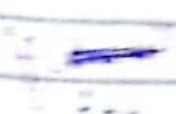
0 < 2 ✓

3 < 7 ✓

for (int j = L; j < H; j++)
{
if (arr[j] < piv)



arr = [1, 2, 3, 4, 5]



t = 0

int t = arr[0]

return

arr[0] = 1

t = 1

int t = arr[1]

return

return arr[0]

return arr[1]

arr[0] = 1

quickSort(arr, 0, arr.length - 1)

quickSort(arr, 0, arr.length - 1)

int pi = part(arr, 0, arr.length - 1)

// again recursively call

return arr;

Question # 07

// Explained in DR:
(DRY - RUN)

Suppose: arr is {3, 2, 7}
n = 3

```
void cs(int arr[], int n)
```

```
{  
    int max = arr[0]
```

max = 3

1) for (—) ✓

if (—) ✗

2 > 3

~~max = arr[0]~~

2) again for (—) ✓

if (—) ✓

7 > 3

{ max = arr[i]; }

max = 7

3) again for (—) ✗

3 < 3 ✗

int count[max+1] = {0};

1) for (int i = 0; i < n; i++)

{
 count[arr[i]]++;

arr[0] = 3

count[3]++; {0, 0, 1, 0, 0, 0, 0}

2) ag count[2]++; {0, 0, 1, 1, 0, 0, 0, 0}

3) ag count[7]++; {0, 0, 1, 1, 0, 0, 0, 1}

```

for (i=0; i<max; i++)
{
    count[i] = 0;
}

```

```

if (count[i] > 0)
{
    prompt;
}

```

// every element = 1

```

for (int i=1; i<max; i++)
{
    count[i] += count[i-1];
}

```

0 += 0 = 0

eg: {0, 0, 1, 1, 0, 0, 0, 1}

ag: {0, 0, 1, 1, 0, 0, 0, 1}

ag: {0, 0, 1, 2, 0, 0, 0, 1}

ag: {0, 0, 1, 2, 2, 0, 0, 1}

ag: {0, 0, 1, 2, 2, 2, 0, 1}

ag: {0, 0, 1, 2, 2, 2, 2, 1}

ag: {0, 0, 1, 2, 2, 2, 2, 3}

ind output[n]; output[9]

```

for (int i=n-1; i>=0; i--)
{
    output[count[i]-1] = arr[i];
    count[i]--;
}

```

eg: for (i=7) { output[count[7]-1] = arr[7] → output[3] = {_, _, _, 7}

count[7]--; → count[7] = 2

eg: for (i=6) { output[count[6]-1] = arr[6] → output[3] = {2, _, _, 7}

count[6]--; → count[6] = 1

Searching - Algorithm

All - Questions

Question # 01

```
int BS(int arr[], int size, int t)
{
    for(int i=0; i<size; i++)
```

```
    {
        if(arr[i]==t)
```

```
        {
            return i;
```

```
        }
```

```
    }
```

```
    return -1;
```

(DRY - RUN)

Suppose:

arr is {1, 2, 7, 9}

target is 7

size = 4

1) for(int i=0; i<4; i++) ✓

if(arr[i]==t) X 1 ≠ 7

2) for() ✓ 0 < 4 ✓

if() X 2 ≠ 7

3) for() ✓ 2 < 4

if() ✓ 7 = 7

return i;

i = 2

Question # 02.

// Just loop $\rightarrow$ Explained in DRY_RUN
(DRY_RUN)

Suppose: arr is = {1, 2, 3, 4, 5}

```
int left = 0;
int Right = size - 1;
```

$$15 - 9 = 4 \quad \boxed{R24}$$

o) while($L \leq R$) ✓

```
int mid = L + (R - L) / 2;
```

0+9/20+20+20
mid 20

```
if (array[mid] == target) {
```

```
if (arr[mid] < target) ✓
```

```
l = mid + 1;
```

L3

→ again while ($L \leq R$)

```
int m2 L+(R-L)/2;
```

$$3 + 1/2 = \text{mid} = 3$$

```
if (arr[mid] == tar) ✓ 4 == 4
{
    return mid;
}
```


Question # 03...

// Just loop $\rightarrow$ Explained in dry-run

(DRY-RUN)

$\rightarrow 0 \rightarrow R = \text{size} - 1 = 6 - 1 = 5$

BSR(int arr[], int L, int R, int T)

if (L > R) $\times$ $L = 0, R = 5$

int mid = $L + (R - L) / 2$; $\frac{0 + 5}{2}$
 $\boxed{\text{mid} = 2}$

Suppose arr is
{1, 2, 3, 4, 5, 6}
target is 4
size = 6

if (arr[mid] == target) $\times$

if (arr[mid] < target) $\checkmark$

return BSR(arr, mid+1, R, T);

now
return to
main or
caller

② Recursion

BSR(int arr[], int L, int R, int T)

?

if (L > R) $\times$

int mid = $L + (R - L) / 2$;

$\frac{3 + 5}{2} =$
mid = 4

if (arr[mid] == T) $\checkmark$

return mid; // return to caller ①